

1. Arquitectura del Código y Flujo de Trabajo

La arquitectura del simulador físico Cosmic-Generator está estructurada para maximizar la modularidad, el rendimiento y la facilidad de uso. El flujo de trabajo o pipeline sigue un modelo secuencial desde la captura de parámetros del usuario hasta la visualización en tiempo real y la síntesis de audio.

1.1. Punto de Entrada: UI y Configuración (`app.py`)

El ciclo de ejecución de la plataforma comienza en el archivo `app.py`. Este script sirve como punto de entrada de la aplicación y utiliza el framework Streamlit para proveer una Interfaz de Usuario (UI) web interactiva. En esta etapa, `app.py` permite al usuario definir las variables globales y parámetros específicos del sistema (tales como constantes físicas, niveles de fricción, propiedades visuales, entre otros). Al iniciar la simulación, los parámetros se empaquetan y se transmiten al siguiente nivel de la arquitectura.

1.2. Orquestación del Sistema (`lanzador.py`)

El módulo `lanzador.py` cumple el rol de orquestador o controlador principal. Recibe los parámetros configurados en la UI e inicializa todas las piezas fundamentales del simulador. Sus responsabilidades incluyen:

- Instanciar el entorno físico.
- Configurar los subsistemas de generación de audio y renderizado visual.
- Disparar el bucle principal de simulación, delegando la carga computacional a los módulos especializados.

De esta forma, se mantiene completamente desacoplada la interfaz web de la lógica pesada del simulador.

1.3. Patrón Productor-Consumidor en Renderizado (`src/render/stable/render_standard.py`)

El rendimiento es crítico en simulaciones físicas de tiempo real. Para asegurar una alta tasa de fotogramas por segundo (FPS), el archivo `src/render/stable/render_standard.py` implementa un modelo de concurrencia basado en el patrón **Productor-Consumidor**, empleando múltiples hilos (threads):

- **Productor (Hilo Físico):** Un hilo en segundo plano se dedica exclusivamente a resolver las ecuaciones matemáticas del simulador. Calcula la cinemática, actualiza posiciones, velocidades y detecta colisiones, produciendo *estados del sistema* que se almacenan de forma segura en una estructura de datos concurrente (cola o buffer).
- **Consumidor (Hilo Gráfico):** El hilo principal de renderizado lee o consume los estados físicos ya calculados que están disponibles en el buffer, para dibujar cada fotograma. Al separar estos dos procesos, el cálculo matemático no bloquea el repintado de la pantalla y la visualización gráfica no ralentiza la física, logrando una aceleración notable de los FPS.

1.4. Organización Modular del Código

El código fuente se organiza de forma rigurosa y cohesiva en distintos directorios bajo `src/`, respetando el principio de responsabilidad única:

- **src/core/**: Contiene el motor numérico y físico del proyecto. Aquí residen los cálculos de dinámica gravitacional, cinemática y detección de colisiones. Este módulo opera con matemática pura y no tiene conocimiento del entorno gráfico ni sonoro.
- **src/audio/**: Módulo encargado de la síntesis y gestión de sonido. Actúa escuchando los eventos disparados por el Core (por ejemplo, el rebote o impacto de una entidad) y los transforma dinámicamente en efectos de audio espaciales o arreglos musicales.
- **src/render/**: Dedicado de forma exclusiva a la representación visual. Recopila la información geométrica del Core y realiza las llamadas a la biblioteca gráfica pertinente para renderizar el entorno y los elementos simulados en la ventana de la aplicación.